MTU Assistant program design document

Version 0.3.0. This document describes the design of MTU Assistant, a campus chatbot for Michigan Technological University. It states the problem the program solves, the constraints that shaped it, how the parts are divided, and how a question becomes an answer.

1. Purpose

MTU Assistant answers student questions about public campus information: news, events, library hours, tutoring, and registration. A student types a question in a browser. The program retrieves the public pages that match and replies with the specific facts, then shows the page titles under the message.

The program does not answer from a general web search, and it does not read login-gated systems. Banweb, Canvas, grades, and personal schedules are out of scope.

2. Design goals

1. Answers name the fact (a time, a place, a step) instead of telling the student to go read a page.
2. Campus text and the answer stay on the machine that runs the server. No hosted chat API is called.
3. The stored pages can be searched by meaning, so a paraphrased question still finds the right section.
4. Pages are collected politely from public sites and replaced when the text changes.
5. One person can run the whole program on a laptop, with an optional shared database later.

3. Constraints

These constraints decide the shape of the program.

Constraint	Design consequence
Prefer an in-house model over sending university text to an	Generation uses Transformers on this machine, or Ollama on localhost. Gemini and

Constraint	Design consequence
open model API	similar clients are not on the answer path.
Choose document, vector, or graph memory	A vector store. Questions need the closest page text in meaning. There is no relationship to traverse, so there is no graph.
Decide whether embeddings are required, and whether the database creates them	Embeddings are required for that search. Chroma and pgvector only store vectors. `app/embed.py` creates them locally.
Consider LangChain or a similar orchestrator	LangChain builds the prompt and calls the local model. Fetching, chunking, and storage stay in this program.
Do not use hosted model APIs; use Transformers or Ollama	`LLM_BACKEND` is `transformers` or `ollama`. LangSmith tracing is forced off so notes are not uploaded.
Public pages only	The source list is a fixed set of public URLs. The scraper identifies itself, honors `robots.txt`, and waits between requests.

4. System context

```
Student browser
      |
      |  GET /          the chat page
      |  POST /chat     question, topic, recent turns
      v
MTU Assistant (one process)
      |
      +-- public MTU sites     read on a schedule, not during the question
      |
      +-- vector store         Chroma files, or Postgres/pgvector
      |
      +-- local models         embeddings, and the answer model
```

The student is the only actor on the normal path. An operator with the admin key can force a refresh. The browser holds the conversation. The server does not keep a transcript.

5. Architecture

The program is a single FastAPI process. A static page is the client. Four pipelines sit behind it: collect, store, retrieve, and answer.

```
app/static/index.html
        |
        v
app/main.py                 HTTP, lifespan, admin check
        |
        +-- app/ingest.py   scheduler and change detection
        |        |
        |        v
        |   app/scraper.py  fetch, extract, crawl
        |
        +-- app/store.py    Chroma or pgvector
        |        |
        |        v
        |   app/embed.py    chunk text and embed it
        |
        +-- app/rag.py      choose sections, answer, check facts
                 |
                 v
             Transformers or Ollama
```

`app/config.py` holds settings and the source list. Nothing else decides which sites are read or which model runs.

Startup does four things in the background, then opens the port:

1. Load the embedding model by encoding a probe sentence.
2. Load the answer model, or build the Ollama client.
3. Register one interval job per source.
4. Start one thread per source for the first read.

The interval jobs are added with `next_run_time=None`. APScheduler treats that as paused, so a later refresh is `POST /ingest` or a process restart. The first read still happens from the startup threads.

One worker is part of the design. The scheduler, the model, and a generation lock all live in that process. More than one web process would run the scheduler more than once and would not share the lock.

6. Module design

6.1 `app/main.py`

The HTTP boundary. It serves the page, checks the admin key, and turns `stream_chat` into server-sent events. Request bodies are Pydantic models. A question is 1 to 2000 characters. A topic is optional. Up to eight prior turns are accepted. Retrieval width `top_k` is 1 to 8, default 4.

CORS is installed only when `CORS_ORIGINS` is set. Empty means the page and the API share an origin.

6.2 `app/config.py`

`Settings` reads `.env`. The source list is code, not configuration, so a new site is a reviewed change.

Each source has a name, a category, a kind, start URLs, and a refresh interval.

Kind	Behavior
`feed`	RSS or Atom. Up to 50 entries are fetched as pages. An HTML landing page may point at a feed, or be crawled.
`page`	Fetch the listed URLs and stop.
`crawl`	Same host, paths matching `link_patterns`, skip common binary files, stop after `MAX_PAGES_PER_SOURCE` (default 60).

Current sources:

Name	Category	Kind	Starts at
`mtu-news`	news	feed	`https://www.mtu.edu/news/`
`mtu-events`	events	page	`https://events.mtu.edu/` and `/calendar`
`library`	library	crawl	`https://www.mtu.edu/library/`
`tutoring`	tutoring	crawl	Learning centers, math center, computing center, writing center, success coaches
`registration`	registration	crawl	`https://www.mtu.edu/registrar/`

6.3 `app/scraper.py`

Turns a source into documents. A document is a URL, title, plain text, category, and source name. The content hash is SHA-256 of the text.

Fetching rules:

- Send `USER_AGENT`.

- Read `robots.txt` per host and skip disallowed URLs.
- Wait `REQUEST_DELAY_S` (default 1 second) between hits to the same host.
- Plain fetches time out at 20 seconds.
- Trafilatura extracts the main text, including tables. If that text is under 200 characters, visible text is used after `script`, `style`, `nav`, `footer`, and `header` are removed.
- Non-HTML responses are not stored.

`render_js` is off. If a source sets it, Playwright renders the page. A missing Playwright install falls back to a plain fetch.

6.4 `app/ingest.py`

One source runs at a time, so the embedder and the database are not driven by overlapping crawls. For each document the store returns `new`, `updated`, or `unchanged`. If the fetch returned at least one page, URLs of that source that were not in the fetch are deleted. An empty fetch does not prune, so a failed request does not wipe the index.

The result is kept in memory as `last_runs` and returned by `/health`. It is not written to disk.

6.5 `app/embed.py`

One sentence-transformer, loaded once under a lock. The default is `sentence-transformers/all-MiniLM-L6-v2`. Vectors are L2-normalized. The same function embeds a page chunk and a question, which is required for cosine search to be meaningful.

Chunking splits on paragraphs, then packs them up to 1200 characters. A paragraph longer than that is sliced with 150 characters of overlap.

This path is not LangChain `HuggingFaceEmbeddings`. That helper strips newlines, which would not match vectors already stored by this encoder.

6.6 `app/store.py` and the backends

`store.py` is a facade. `VECTOR_BACKEND=chroma` uses `store_chroma.py`. `pgvector` uses `store_pg.py`. Callers see `upsert_doc`, `prune_source`, `search`, and `stats`.

Chroma is a persistent collection named `mtu` with cosine HNSW, under `CHROMA_DIR`. It needs no extra service.

pgvector creates a `chunks` table and an HNSW cosine index on startup. It is the design for more than one server. Switching backends does not copy rows. The index has to be filled again.

Search embeds the query, takes the nearest `k` chunks, and can filter on `category`. The reported score is cosine similarity.

A chunk's metadata is:

Field	Meaning
`url`, `title`	Page the text came from
`category`	`news`, `events`, `library`, `tutoring`, or `registration`
`source`	Source name, used when pruning
`content_hash`	Hash of the whole page, so a change replaces every chunk of that URL
`chunk`	Index within the page
`ingested_at`	Unix time

The chunk text lives in the vector store. There is no second document database.

6.7 `app/rag.py`

This is the answer design.

1. Search with at least eight chunks so duplicate URLs do not use up the whole result.
2. If a topic filter matches nothing, search again with no topic.
3. Split each chunk into sections. A new section starts at a short heading that follows a line with a time, "Closed", or "24/7".
4. Embed the question and each section. Keep sections near the best score. Drop a weak best score (below 0.48) instead of answering from an unrelated page.
5. Drop other services on a hours page (coffee shop, IT desk, testing center) unless the question names them. Drop a schedule whose every timed row is a date already past. Keep overnight access when the question is about hours or the library.
6. A hours or overnight question is answered by formatting those rows. The model is not asked to rewrite a table.
7. Any other question is sent to the local model with only the selected sections. Decoding is greedy. If the reply contains a time range, weekday pairing, or phone number that is not in those sections, the reply is discarded and the section text is sent instead.

8. The page title and URL are a separate event. They are not part of the answer sentence.

LangChain's role is the prompt template, the local runnable, and a string parser. The Transformers runnable applies the model's chat template and streams tokens with `TextIteratorStreamer` on a daemon thread. Ollama uses `ChatOllama` at `OLLAMA_BASE_URL` with temperature 0.

The default answer model is `Qwen/Qwen2.5-0.5B-Instruct` in float32 on CPU, with at most 320 new tokens. One generation holds a lock, so a second question waits.

6.8 `app/static/index.html`

The whole client. There is no build step.

The page sends `POST /chat` with the text, the selected topic (`news`, `events`, `library`, `tutoring`, `registration`, or null), and the last six successful turns. It reads server-sent events, appends token text, and renders a small Markdown subset. Source titles become links. Enter sends. Shift+Enter inserts a line. The send button stays disabled until the stream ends. Colors follow the operating system theme.

History is trimmed in the browser (assistant text to 1500 characters) and again on the server (each turn to 700 characters inside the prompt, at most six turns).

7. Data design

Two kinds of state exist.

Durable. Page chunks and their vectors, in `chroma_data/` or in Postgres. This is derived data. It can be deleted and built again from the source list. It is not committed.

Process memory. The loaded models, `last_runs`, the robots cache, and the generation lock. A restart clears them and reloads models and pages.

Not stored. Chat transcripts, student identity, and cookies. The admin key and any unused provider keys stay in `.env`, which is gitignored.

The content hash is the change detector. Equal hash means the chunks stay. A new hash deletes every chunk for that URL and inserts the new ones, so removed paragraphs disappear.

8. Process design

8.1 Collecting pages

```
startup thread or POST /ingest
        |
        v
fetch_source
        |
        +-- robots.txt and delay
        +-- feed, page, or crawl
        v
extract text and hash
        |
        +-- hash matches stored hash --> unchanged
        +-- hash differs --> delete old chunks, chunk, embed, insert
        v
if the fetch returned pages: delete URLs no longer seen
```

News is designed to be read often (15 minutes) and the registrar rarely (12 hours). Because the interval jobs are paused, those numbers describe the intended period, and a running process refreshes when ingest is called or the process starts.

8.2 Answering

```
POST /chat
    |
    v
embed question --> nearest chunks --> sections --> re-rank
    |
    +-- nothing close enough --> "no campus page" and stop
    |
    +-- hours or overnight --> format the current rows
    |
    +-- otherwise --> local model on those sections
                          |
                          +-- facts match the sections --> send the reply
                          +-- a time or number was invented --> send the
section text
    |
    v
sources, then done
```

The hours path does not wait for the answer model. That is deliberate: a small model was blending Monday–Thursday library hours with other tables on the same page.

8.3 Event stream

Each event is one JSON object prefixed with `data:`.

type	When
`status`	Retrieval has started
`token`	Answer text. A grounded reply may arrive as one piece.
`sources`	Pages used, each with a title and URL
`done`	The stream is finished
`error`	The model failed before a reply was chosen

9. Interface design

Method	Path	Who	Result
`GET`	`/`	student	Chat page
`POST`	`/chat`	student	Event stream
`GET`	`/health`	operator	Chunk count, backend, model, last ingest
`GET`	`/sources`	operator	The source list
`GET`	`/search`	operator	Raw nearest chunks
`POST`	`/ingest`	operator, `X-API-Key`	Refresh every source
`POST`	`/ingest/{name}`	operator, `X-API-Key`	Refresh one source

`/health` and `/search` are unauthenticated and can return stored page text. The process is designed to bind to localhost unless it is placed behind a proxy. The admin key gates only ingest, because a refresh spends a long time fetching and embedding.

10. Decisions and alternatives

Vector memory, not a document store and not a graph. A keyword store misses "when is the library open" against a page titled with hours. A graph would fit advisor-to-major links or course prerequisites. This program does not have those edges. The chunks themselves are the documents, stored inside the vector database, so a second document store would duplicate them.

Local embeddings, not database-side embeddings. The search has to use the same vector space as the stored pages. Keeping that in one Python module makes the model explicit and keeps page text off a hosted embedding API.

LangChain at the prompt boundary only. The scraper and the store already have the metadata the product needs (URL, category, hash). Wrapping them in a framework retriever would not improve the hours-section logic, which is the part that keeps schedules accurate.

Copy a schedule; generate the rest. Free generation from a 0.5B model rewrote clock times. Tables are formatted from the selected section. Prose still goes through the local model, then a check rejects times and phone numbers that were not in the notes.

Chroma by default, pgvector as a seam. Laptops should start with no database server. The facade exists so a shared deployment can switch without rewriting retrieval.

Single page, no frontend build. The client is one HTML file. That matches a local campus tool and keeps the event parser next to the composer.

Rejected: hosted chat models. They were faster and more fluent. They send campus notes off the machine, and the available models were returning quota and availability errors. The assignment constraint is the stronger reason.

Rejected: a graph database, and rejected: letting Chroma or pgvector embed text. Both conflict with the memory and embedding decisions above.

11. Concurrency, failures, and security

- Ingest is single-threaded through one lock. Startup threads queue on it.
- Generation is single-threaded through a second lock. Embedding has its own init lock.
- A fetch error logs and skips that URL. A source that returns no documents does not prune.
- A model failure before any reply becomes an `error` event. An Ollama connection failure says to start Ollama and pull `LLM_MODEL`.
- `.env` is not part of the program source. `ADMIN_API_KEY` should be a long random string before the port is reachable by anyone else.
- The scraper must not be pointed at authenticated systems.

12. Deployment design

Development is `uvicorn app.main:app` on `127.0.0.1:8000`, one process, no reload. Dependencies are `requirements.txt`. Postgres and Playwright are optional (`requirements-optional.txt`, `docker-compose.yml`).

The first question after a cold start pays for model load. Later questions in that process reuse the weights. Hours questions do not wait on the answer model once embeddings are loaded.

Changing `EMBED_MODEL` or the chunk size invalidates stored vectors. The index has to be rebuilt. Changing `LLM_MODEL` does not.

13. Known limits

- The default 0.5B model can still misstate a prose answer when the mistake is not a time or a phone number. The fallback then shows the section instead of a polished paragraph.
- Interval refresh does not fire by itself until the paused jobs are given a next run time.
- The events pages are a snapshot of the public calendar, not every future event.
- `/search` embeds on the request thread. A cold embedding model blocks that call.
- The design assumes one server process.